

Deep Learning
Project 3 report
Generative Models (cats & dogs datasets)

Bogumiła Okrojek

Katsiaryna Bokhan

May 2026

Abstract

Contents

1	Problem Description	3
1.1	Task formulation	3
1.2	Datasets and preprocessing	3
2	Theoretical Introduction	3
2.1	WGAN-GP (Wasserstein GAN with Gradient Penalty)	3
2.2	VQ-VAE (Vector Quantized Variational Autoencoder)	3
2.3	StyleGAN2-ADA	4
2.4	FID (Fréchet Inception Distance)	4
3	Literature Review	4
4	Experiments	4
4.1	WGAN-GP Training Experiment – cats dataset	5
4.2	VQ-VAE Training Experiment – cats dataset	5
4.3	StyleGAN2-ADA Training – cats dataset	5
4.4	Comparison of three best models – cats dataset	5
4.5	StyleGAN2-ADA Training Experiment – cats & dogs dataset	5
5	Results	5
5.1	WGAN-GP Training Experiment – cats dataset	5
5.2	VQ-VAE Training Experiment – cats dataset	5
5.3	StyleGAN2-ADA Training - cats dataset	5
5.4	Comparison of three best models – cats dataset	5
5.5	StyleGAN2-ADA Training Experiment – cats & dogs dataset	5
6	Conclusion	5
7	Instruction on reproducing the results?	5

1 Problem Description

1.1 Task formulation

The main purpose of this project is to train and compare different generative models on the cats (and cats vs. dogs) datasets. Our version of the project focuses on three different models: WGAN-GP [1], VQ-VAE [6] and StyleGAN2-ADA [4]. All of them are trained from zero, without any pretraining. While StyleGAN2-ADA was directly taken from the original repository, WGAN-GP and VQ-VAE were implemented from scratch by us.

We evaluate the models using the Fréchet Inception Distance (FID) metric, as well as the ResNet50-based species scores, which are the summed softmax probability over ImageNet cat breeds (classes 281–285) and, where relevant, dog breeds (151–268). Except quantitative evaluation, we also inspect generated images qualitatively.

1.2 Datasets and preprocessing

We use the Kaggle Cat Dataset [3] for the main experiments and the Kaggle Dogs vs. Cats dataset [2] for the mixed-class extension.

Cats dataset The Cats dataset originally contains 9997 images of cats, all in different poses, backgrounds, lighting, and resolutions. Before training, we resize the images to 128×128 pixels and apply random horizontal flips as on-the-fly augmentation.

Dogs vs. Cats dataset The Dogs vs. Cats dataset contains images from both the original Kaggle train and test folders. We pool all cat and dog images into one unlabeled set (class labels are not used during training) so that the model can learn both categories from data alone. Images are preprocessed the same way: resize to 128×128 and random horizontal flips.

Train / validation / test splits For all models that use `cat_datasets.py`, we split the cat images once with a fixed random seed (42) into three disjoint subsets. We chose small validation and test sets on purpose, so that as many images as possible remain for training:

- **Cats:** 500 validation and 500 test images; the remaining $\approx 9,000$ images are used for training (9997 total).
- **Cats & dogs:** 4,000 validation and 4,000 test images; the remaining $\approx 29,500$ images are used for training ($\approx 37,500$ images in total after merging train and test folders). StyleGAN2-ADA on the mixed dataset uses `max_size= 29,440` training images.

For FID and ResNet50 evaluation we compare generated samples against the held-out **test** set (typically 500 real test images for cats, and the same convention where applicable). Early stopping and checkpoint selection use the **validation** split.

2 Theoretical Introduction

2.1 WGAN-GP (Wasserstein GAN with Gradient Penalty)

2.2 VQ-VAE (Vector Quantized Variational Autoencoder)

Variational Autoencoders (VAEs) learn a continuous latent representation, but the Gaussian posterior and continuous latents often produce blurry reconstructions. The Vector Quantized VAE (VQ-VAE) [6] replaces continuous latents with **discrete** codes drawn from a finite **codebook**, yielding a compressed representation that can be modeled as a sequence of categorical variables and decoded into sharper images.

Architecture. Given an input image $\mathbf{x}$, an encoder E_ϕ maps it to a latent feature map $\mathbf{z}_e = E_\phi(\mathbf{x})$, typically with lower spatial resolution than $\mathbf{x}$. A vector quantizer Q compares each spatial feature vector $\mathbf{z}_e^{(i)}$ to entries of a learned codebook $\mathcal{E} = \{\mathbf{e}_1, \dots, \mathbf{e}_K\}$, $\mathbf{e}_k \in \mathbb{R}^D$, and replaces it by the nearest code:

$$k^* = \arg \min_k \left\| \mathbf{z}_e^{(i)} - \mathbf{e}_k \right\|_2^2, \quad \mathbf{z}_q^{(i)} = \mathbf{e}_{k^*}.$$

The collection of indices $\{k^*\}$ forms a discrete latent grid. A decoder D_θ reconstructs the image from the quantized map: $\hat{\mathbf{x}} = D_\theta(\mathbf{z}_q)$. Thus the model learns an autoencoder whose bottleneck is discrete rather than continuous.

Loss function. Following [6], let $\mathbf{z}_e$ be the encoder output and $\mathbf{z}_q$ the quantized latent obtained by replacing each spatial feature vector with its nearest codebook entry $\mathbf{e}$ (so $\mathbf{z}_q$ and $\mathbf{e}$ coincide at each location after quantization). Training minimizes

$$\mathcal{L} = \underbrace{-\log p(\mathbf{x} \mid \mathbf{z}_q)}_{\text{reconstruction}} + \underbrace{\|\text{sg}[\mathbf{z}_e] - \mathbf{e}\|_2^2}_{\text{codebook}} + \beta \underbrace{\|\mathbf{z}_e - \text{sg}[\mathbf{e}]\|_2^2}_{\text{commitment}}, \quad (1)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator and $\beta > 0$ is a scalar weight. The reconstruction term $-\log p(\mathbf{x} \mid \mathbf{z}_q)$ encourages the decoder to recover the input from quantized codes; with a Gaussian observation model it corresponds to a squared reconstruction error $\|\mathbf{x} - D_\theta(\mathbf{z}_q)\|_2^2$.

The **codebook** term $\|\text{sg}[\mathbf{z}_e] - \mathbf{e}\|_2^2$ trains the codebook entries $\mathbf{e}_k$, not the encoder. Here $\text{sg}[\mathbf{z}_e]$ means “treat $\mathbf{z}_e$ as a fixed target”: after quantization, each spatial location was assigned some code k^* , and this term penalizes the distance between that code vector $\mathbf{e}_{k^*}$ and the encoder vector $\mathbf{z}_e$ that selected it. Minimizing it pulls each used $\mathbf{e}_k$ closer to the $\mathbf{z}_e$ vectors that keep choosing k , so codebook vectors track where the encoder actually maps images.

The **commitment** term $\|\mathbf{z}_e - \text{sg}[\mathbf{e}]\|_2^2$ does the opposite: $\mathbf{e}$ is fixed (stop-gradient) and the encoder is trained so that $\mathbf{z}_e$ lies near the code it was quantized to, instead of hopping between distant codes.

Codebook collapse. Such a problem occurs when only a small subset of the K codebook vectors is ever used, so the discrete bottleneck effectively has much lower capacity than designed. It is usually mitigated by the commitment part of the loss and by monitoring codebook utilization during training.

Codebook update via EMA. The original VQ-VAE updates the codebook via the codebook loss presented above; **exponential moving average (EMA)** updates were introduced later in VQ-VAE-2 [5]. After each batch, every code $\mathbf{e}_k$ is set to a smoothed average of the encoder vectors that were assigned to k , so the codebook tracks recent data without a separate codebook gradient term (training then uses reconstruction and commitment only).

Generative modeling. VQ-VAE by itself is primarily a learned compressor and decompressor. To **generate** new data, a second model is trained on the discrete latents to approximate $p(\mathbf{z}_q)$ – for example a PixelCNN or autoregressive transformer that predicts the next code index given previous indices. Sampling proceeds by drawing a latent index sequence from this prior and decoding it with D_θ .

2.3 StyleGAN2-ADA

2.4 FID (Fréchet Inception Distance)

3 Literature Review

4 Experiments

Unfortunately, due to time and computational constraints, we did not have resources to train the same model for a couple of times with different seeds. Instead we focused on running each considered configuration for a reasonable number of epochs (but only once). When we performed experiments focused on image generation, we generated 500-1000 images and for each metric the average value was computed.

- 4.1 WGAN-GP Training Experiment – cats dataset
- 4.2 VQ-VAE Training Experiment – cats dataset
- 4.3 StyleGAN2-ADA Training – cats dataset
- 4.4 Comparison of three best models – cats dataset
- 4.5 StyleGAN2-ADA Training Experiment – cats & dogs dataset

5 Results

- 5.1 WGAN-GP Training Experiment – cats dataset
- 5.2 VQ-VAE Training Experiment – cats dataset
- 5.3 StyleGAN2-ADA Training - cats dataset
- 5.4 Comparison of three best models – cats dataset
- 5.5 StyleGAN2-ADA Training Experiment – cats & dogs dataset

6 Conclusion

7 Instruction on reproducing the results?

References

- [1] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [2] Kaggle. Dogs vs. cats. <https://www.kaggle.com/competitions/dogs-vs-cats/>, 2013.
- [3] Kaggle. Cat dataset. <https://www.kaggle.com/datasets/crawford/cat-dataset>, 2018.
- [4] Tero Karras, Miika Aittala, Janne Hellsten, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Training generative adversarial networks with limited data. *Advances in Neural Information Processing Systems*, 33:12104–12114, 2020.
- [5] Ali Razavi, Aaron van den Oord, and Oriol Vinyals. Generating diverse high-fidelity images with VQ-VAE-2. *Advances in Neural Information Processing Systems*, 32, 2019.
- [6] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.